

Problem Statement

QuickBite – Food Ordering Mobile Application

SE2020 – Web and Mobile Technologies · Abisheak S (IT21244384)

Background and problem

Small restaurants and campus canteens around Malabe still take most food orders by phone or at the counter. Staff write orders on paper, customers have no idea whether a dish is still available, and it is common for the kitchen to accept more orders for a popular dish than it can actually prepare. When a dish sells out, customers are phoned back, orders are changed by hand and the bill is recalculated manually, which leads to mistakes and unhappy customers. Customers also cannot see what stage their order is at without calling the restaurant.

QuickBite is a mobile application that solves this for a single restaurant. Customers register, browse a live menu with photos, prices and remaining portions, place an order and follow it through the kitchen (Pending → Preparing → Ready → Completed). The restaurant administrator manages the menu from the same app and moves orders through the workflow.

The key problem the system solves is keeping the menu and the orders **consistent**: an order can never take more portions than are left, a dish is automatically marked unavailable when it sells out, cancelling an order puts the portions back, and the order total is always calculated by the server from the stored prices so it cannot be manipulated from the phone.

Objectives

- Secure registration and login with bcrypt password hashing and JWT-based sessions.
- Full CRUD for **Menu items** (primary entity), including a photo upload.
- Full CRUD for **Orders** (related entity) that reference menu items, with a status workflow.
- Business rules for stock reservation, automatic sold-out, server-side totals and valid status changes.
- A React Native app with navigation, validated forms and loading / empty / error states, using only data from the hosted API.
- Backend deployed on Render and connected to MongoDB Atlas.

Entities

Entity	Key fields	Operations
MenuItem (primary)	name, description, category, price, stockQuantity, isAvailable, image	Create, read all, read one, update, delete; image upload
Order (related)	user → User, items[].menuItem → MenuItem, quantity, unitPrice, totalAmount, deliveryAddress, notes, status	Create, read all, read one, update (Pending), delete; status change and cancel
User	name, email, phone, password (bcrypt), isAdmin	Register, login, current user

Business rules

- An order cannot take more portions than are in stock; stock is reserved atomically and multi-item orders are all-or-nothing.
- A dish that reaches 0 portions becomes unavailable automatically; cancelling or deleting a Pending order returns the portions.
- The order total is calculated on the server from stored prices and recalculated when a Pending order is edited.

- Orders follow Pending → Preparing → Ready → Completed, and can be cancelled only from Pending (customer) or Pending/Preparing (admin).
- A menu item that is part of an active order cannot be deleted.